P3351R1

`views::scan`

Published Proposal, 2024-10-09

This version:
https://wg21.link/P3351R1

Author:
Yihe Li

Audience:
SG9

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:
C++26

Source:
github.com/Mick235711/wg21-papers/blob/main/P3351/P3351R1.bs

Issue Tracking:
GitHub Mick235711/wg21-papers

Table of Contents

This paper proposes the `views::scan` range adaptor family, which takes a range and a function that takes the current element *and* the current state as parameters. Basically, `views::scan` is a lazy view version of `std::inclusive_scan`, or `views::transform` with a stateful function.

To make common usage of this adaptor easier, this paper also proposes two additional adaptor that further arguments `views::scan`'s functionality:

- `views::partial_sum`, which is `views::scan` with the binary function defaulted to `+`.
- `views::prescan`, which is `views::scan` with an initial state seed.

The `views::scan` adaptor is classified as a Tier 1 item in the Ranges plan for C++26 ([P2760R1]).

§ 1. Revision History

§ 1.1. R1 (2024-10 pre-Wrocław Mailing)

- Several wording fixes:
  - Added the missing `noexcept` to `end()`.
  - Refactored the constraints of `scan_view` out into its own concept, such that it tests for both assignability from `range_reference_t<R>` and the invoke result of `f`.
  - Replace `regular_invocable` with `invocable`.
  - Pass by move in `scan_view`'s constructor and when invoking the function.
- Rebase onto latest draft [N4988].

§ 1.2. R0 (2024-07 post-St. Louis Mailing)

- Initial revision.

§ 2. Motivation

The motivation for this view is given in [P2760R1] and quoted below for convenience:

> If you want to take a range of elements and get a new range that is applying `f` to every element, that's `transform(f)`. But there are many cases where you need a `transform` to that is stateful. That is, rather than have the input to `f` be the current element (and require that `f` be `regular_invocable`), have the input to `f` be both the current element and the current state.
>
> For instance, given the range `[1, 2, 3, 4, 5]`, if you want to produce the range `[1, 3, 6, 10, 15]` - you can't get there with `transform`. Instead, you need to use `scan` using + as the binary operator. The special case of `scan` over + is partial_sum.
>
> One consideration here is how to process the first element. You might want `[1, 3, 6, 10, 15]` and you might want `[0, 1, 3, 6, 10, 15]` (with one extra element), the latter could be called a `prescan`.

This adaptor is also present in ranges-v3, where it is called `views::partial_sum` with the function parameter defaulted to `std::plus{}`. However, as [P2760R1] rightfully pointed out, `partial_sum` is probably not suitable for a generic operation like this that can do much more than just calculating

partial sum, similar to how `accumulate` is not a suitable name for a general fold. Therefore, the more generic `scan` name is chosen to reflect the nature of this operation. (More discussion on the naming are present in the later sections.)

§ 3. Design

§ 3.1. Why Three Adaptors?

An immediately obvious question is why choose three names, instead of opting for a single `views::scan` adaptor with overloads that take initial seed and/or function parameter? Such a design will look like this (greatly simplified):

```
struct scan_closure
{
    template<ranges::input_range Rng, typename T, std::copy_constructible F
    requires /* ... */
    constexpr operator()(Rng&& rng, const T& init, Func func = {})
    { /* ... */ }

    template<ranges::input_range Rng, std::copy_constructible Fun = std::p
    requires /* ... */
    constexpr operator()(Rng&& rng, Func func = {})
    { /* ... */ }
};
inline constexpr scan_closure scan{};
```

First of all, this will definitely cause some confusion to the users, due to the fact that a generic name like `scan` defaults to `std::plus` as its function parameter:

```
vec | views::scan // what should this mean?
```

This is similar to the scenario encountered by `ranges::fold` ([P2322R6]), where despite the old algorithm `std::accumulate` took `std::plus` as the default function parameter, `ranges::fold` still choose to not have a default. The author feels that the same should be done for `views::scan` (and introduce a separate `views::partial_sum` alias that more clearly convey the intent).

However, even put aside the function parameter, why cannot the with-initial-seed version overloads with the use-first-element version?

Unfortunately, this still does not work due to the same kind of ambiguity that caused `views::join_with` ([P2441R2]) to choose a different name instead of overload with `views::join`. Specifically, imagine someone writes a custom `vector` that replicates all the original interface, but introduced `operator+` to mean range concatenation and broadcast:

```
template<typename T>
struct my_vector : public std::vector<T>
{
    using std::vector<T>::vector;

    // broadcast: [1, 2, 3] + 10 = [11, 12, 13]
    friend my_vector operator+(my_vector vec, const T& value)
    {
        for (auto& elem : vec) elem += value;
        return vec;
    }
    friend my_vector operator+(const T& value, my_vector vec) { /* Same */

    // range concatenation: [1, 2, 3] + [4, 5] = [1, 2, 3, 4, 5]
    friend my_vector operator+(my_vector vec, const my_vector& vec2)
    {
        vec.append_range(vec2);
        return vec;
    }
    // operator+= implementation omitted
};
```

Although one could argue that this is a misuse of `operator+` overloading, this is definitely plausible code one could write. Now consider:

```
my_vector<int> vec{1, 2, 3}, vec2{4, 5};
views::partial_sum(vec); // [1, 3, 6]
vec2 | views::partial_sum(vec);
// [[1, 2, 3], [5, 6, 7], [10, 11, 12]] (!!)
```

The second invocation, `vec2 | views::partial_sum(vec)`, is equivalent to `partial_sum(vec2, vec)`, therefore interpreted as "using `vec` as the initial seed, and add each element of `vec2` to it". Unfortunately, we cannot differentiate the two cases, since they both invoke `partial_sum(vec)`. This ambiguity equally affects `views::scan`, since `scan(vec, plus{})` and `vec2 | scan(vec, plus{})` are also ambiguous.

There are several approaches we can adopt to handle this ambiguity:

Option 1: Bail out. Simply don't support the initial seed case, or just declare that anything satisfy `range` that comes in the first argument will be treated as the input range.

- Pros: No need to decide on new names.
- Cons: Losing a valuable use case that was accustomed by users since `std::exclusive_scan` exists. If the "declare" option is chosen, then potentially lose more use case like `my_vector` and cause some confusion.

Option 2: Reorder arguments and bail out. We can switch the function and the initial seed argument, such that the signature is `scan(rng, func, init)`, and declare that anything satisfy `range` that comes in the first argument will be treated as the input range.

- Pros: No need to decide on new names.
- Cons:
  - Still have potential of conflict if a class that is both a range and a binary functor is passed in
  - Cannot support `partial_sum` with initial seed (discard or need new name)
  - Inconsistent argument order with `ranges::fold`

Option 3: Choose separate name for `scan` with and without initial seed.

- Pros: No potential of conflicts
- Cons: Need to decide on 1-2 new names and more wording effort

The author prefers Option 3 as it is the least surprising option that has no potential of conflicts. For now, the author decides that `scan` with an initial seed should be called `views::prescan`, as suggested in [P2760R1], and `partial_sum` should not support initial seed at all. The rationale for this decision is that people who want `partial_sum` with initial seed can simply call `prescan(init, std::plus{})`, so instead of coming up a name that is potentially longer and harder to memorize the author felt that this is the best approach.

More alternative names are suggested in later sections.

§ 3.2. Prior Art

The scan adaptor/algorithm had made an appearance in many different libraries and languages:

- range-v3 has a `views::partial_sum` adaptor that don't take initial seeds, but takes arbitrary function parameter (defaults to `+`).
- `tl::ranges` also has an implementation of `views::partial_sum`.
- Python has an `itertools.accumulate` algorithm that optionally takes initial seeds (with a named argument), and takes arbitrary function parameter (defaults to `+`).
    - Furthermore, NumPy provides `cumsum` algorithm that returns a partial sum (without function parameter), and an `ufunc.accumulate` algorithm that performs arbitrary scans with arbitrary function parameter that have no default. Neither of those algorithms takes an initial seed.
- Rust has an `iter.scan` adaptor that takes initial seeds and arbitrary function parameters (no defaults provided).

Summarized in a table:

Library	Signature	Function	With Default	Initial Seed
range-v3	`partial_sum(rng[, fun])`	✅	✅	❌
Python `itertools`	`accumulate(rng[, fun[, init=init]])`	✅	✅	✅
NumPy	`cumsum(rng)`	❌	N/A	❌
	`<func>.accumulate(rng)`	✅	❌	❌
Rust	`iter.scan(init, func)`	✅	❌	✅
Proposed	`scan(rng, func)`	✅	❌	❌
	`prescan(rng, init, func)`	✅	❌	✅
	`partial_sum(rng)`	❌	N/A	❌

§ 3.3. Alternative Names

The author thinks `views::scan` and `views::partial_sum` are pretty good names. The former have prior example in `std::[in,ex]clusive_scan` and in Rust, and is a generic enough name that will not cause confusion. The latter also have prior example in `std::partial_sum`, and partial sum is definitely one of the canonical terms of describing this operation.

Alternative names considered for `views::scan`: (in order of decreasing preference)

- `views::partial_fold` (suggested by [P2214R2], and makes the connection with `ranges::fold` clear): Pretty good name, since `scan` is just a `fold` with intermediate state

saved. However, the correct analogy is actually `ranges::fold_left_first`, and `ranges::fold_left` actually corresponds to `views::prescan`, which the author felt may cause some confusion.

- `views::accumulate`, `views::fold`, `views::fold_left`: The output is a range instead of a number, so using the same name probably is not accurate. The latter two also suffer from the same displaced correspondence problem.

Alternative names considered for `views::partial_sum`: (in order of decreasing preference)

- `views::prefix_sum`: Another canonical term for this operation, but doesn't have prior examples.
- `views::cumsum` or `views::cumulative_sum`: Have prior example in NumPy, but in the spirit of Ranges naming we probably need to choose the latter which is a bit long.

Alternative naming schemes for all 3 or 4 adaptors proposed:

Option A: Name `views::scan` as `views::inclusive_scan`, and `prescan` as `exclusive_scan`. This will make the three adaptors have the same name as the three algorithms already existed in `<numerics>`, which may seems a good idea at first glance. However, `std::[in,ex]clusive_scan`, despite having a generic name, actually requires its function parameter to be associative (or, more precisely, allow for arbitrary order of executing the function on elements), which `scan` or `prescan` does not. So the author felt that reusing the same name may cause some misuse of algorithms.

Option B: Name `views::scan` as `views::scan_first`, and `prescan` as `scan`. This will make the naming consistent with the `ranges::fold` family, but penalize the more common form of without-initial-seed by making it longer and harder to type. This option also have the advantage of being able to spell `partial_sum_first` and `partial_sum` as the two form of partial sums, instead of being forced to discard one.

Option C: Keep `views::scan`, but name `prescan` as `scan_init` (meaning providing an init parameter). Does not penalize the common case, but also inconsistent with `ranges::fold`. This option also have the advantage of being able to spell `partial_sum` and `partial_sum_init` as the two form of partial sums, instead of being forced to discard one.

Overall, the author don't feel any of these options as particularly intriguing, and opt to propose the original naming of `scan`, `prescan` and `partial_sum`.

§ 3.4. Left or Right Fold?

Theoretically, there are two possible direction of scanning a range:

```
// rng = [x1, x2, x3, ...]
prescan_left(rng, i, f) // [i, f(i, x1), f(f(i, x1), x2), ...]
prescan_right(rng, i, f) // [i, f(x1, i), f(x2, f(x1, i)), ...]
```

Both are certainly viable, which begs the question: Should we provide both?

On the one hand, `ranges::fold` provided both the left and the right fold version, despite the fact that right fold can be simulated by reversing the range and the function parameter order. However, here, the simulation is even easier: just reversing the order of the function parameters will turn a left scan to a right scan.

Furthermore, all of the mentioned prior arts perform left scan, and it is hard to come up with a valid use case of right scan that cannot be easily covered by left scan. Therefore, the author only proposes left scan in this proposal.

§ 3.5. More Convenience Aliases

Obviously, there are more aliases that can be provided besides `partial_sum`. The three most useful aliases are:

```
std::vector<int> vec{3, 4, 6, 2, 1, 9, 0, 7, 5, 8}
partial_sum(vec) // [3, 7, 13, 15, 16, 25, 25, 32, 37, 45]
partial_product(vec) // [3, 12, 72, 144, 144, 1296, 0, 0, 0, 0]
running_min(vec) // [3, 3, 3, 2, 1, 1, 0, 0, 0, 0]
running_max(vec) // [3, 4, 6, 6, 6, 9, 9, 9, 9, 9]
```

Which are the results of applying `std::plus{}`, `std::multiplies{}`, `ranges::min`, and `ranges::max`.

Looking at the results, these are certainly very useful aliases, even as useful as `partial_sum`. However, as stated above, all of these three aliases can be achieved by simply passing the corresponding function object (currently, `min`/`max` doesn't have a function object, but `ranges::min` and `ranges::max` will be useable after [P3136R0] landed) as the function parameter, so I'm not sure it is worth the hassle of specification. Therefore, currently this proposal do not include the other three aliases, but the author is happy to add them should SG9/LEWG request.

As for `partial_sum` itself, there are several reasons why this alias should be added, that is certainly stronger than the case for `product`, `min` and `max`:

- All existing implementation of scan-like algorithm defaults the function argument to `+` whenever there is a default.
- `std::partial_sum` exists
- Partial sum is one of the most studied and used concept in programming, arguably even more useful than running min/max.

§ 3.6. Range Properties

All three proposed views are range adaptors, i.e. can be piped into. `partial_sum` is just an alias for `scan(std::plus{})`, so it will not be mentioned in the below analysis.

§ 3.6.1. Reference and Value Type

Consider the following:

```
std::vector<double> vec{1.0, 1.5, 2.0};
vec | views::prescan(1, std::plus{});
```

Obviously, we expect the result to be `[1.0, 2.0, 3.5, 5.5]`, not `[1, 2, 3, 5]`, therefore for `prescan` we cannot just use the initial seed's type as the resulting range's reference/value type.

There are two choices we can make: (for input range type `Rng`, function type `F` and initial seed type `T`)

1. Just use the reference/value type of the input range. This is consistent with `std::partial_sum`. (range-v3 also chose this approach, using the input range's value type as the reference type of `scan_view`)
2. Be a bit clever, and use `remove_cvref_t<invoke_result_t<F&, T&, ranges::range_reference_t<Rng>>>`. In other words, the return type of `func(init, *rng.begin())`.

Note that the second option don't really covers all kind of functions, since it is entirely plausible that `func` will change its return type in every invocation. However, this should be enough for nearly all normal cases, and is the decision chosen by [P2322R6] for `ranges::fold_left[_first]`. For

`views::scan`, this approach can also work, by returning the return type of `func(*rng.begin(), *rng.begin())`.

Although the second choice is a bit complex in design, it also avoids the following footgun:

```
std::vector<int> vec{1, 4, 2147483647, 3};
vec | views::prescan(0L, std::plus{});
```

With the first choice, the resulting range's value type will be `int`, which would result in UB due to overflow. With the second choice the resulting value type will be `long` which is fine. (Unfortunately, if you used `views::scan` here it will still be UB, and that cannot be fixed.)

The second choice also enables the following use case:

```
// Assumes that std::to_string also has an overload for std::string that ju
std::vector<int> vec{1, 2, 3};
vec | views::prescan("2", [](const auto& a, const auto& b) { return std::to
// ["2", "21", "212", "2123"]
```

Given that `ranges::fold_left[_first]` chose the second approach, the author thinks that the second approach is the correct one to pursue.

§ 3.6.2. Category

At most forward. (In other words, forward if the input range is forward, otherwise input.)

The resulting range cannot be bidirectional, since the function parameter cannot be applied in reverse. A future extension may enable a `scan` with both forward and backward function parameter, but that is outside the scope of this paper.

§ 3.6.3. Common

Never.

This is consistent with range-v3's implementation. The reasoning is that each iterator needs to store both the current position and the current partial sum (generalized), so that the `end()` position's iterator is not readily available in O(1) time.

3.6.4. Sized

If and only if the input range is sized.

For `views::scan`, the size is always equal to the input range's size. For `views::prescan`, the size is always equal to the input range's size plus one.

3.6.5. Const-Iterable

Similar to `views::transform`, if and only if the input range is const-iterable and `func` is `const`-invocable.

3.6.6. Borrowed

Never.

At least for now. Currently, `views::transform` is never borrowed, but after [P3117R0] determined suitable criteria for storing the function parameter in the iterator, it can be conditionally borrowed.

Theoretically, the same can be applied to `scan_view` to make it conditionally borrowed by storing the function and the initial value inside the iterator. However, the author would like to wait until [P3117R0] lands to make this change.

3.7. Feature Test Macro

This proposal added a new feature test macro `__cpp_lib_ranges_scan`, which signals the availability of all three adaptors.

An alternate design is to introduce 3 macros, but the author felt such granularity is not needed.

3.8. Freestanding

[P1642R11] included nearly everything in `<ranges>` into freestanding, except `views::istream` and the corresponding view types. However, range adaptors added after [P1642R11], like `views::concat` and `views::enumerate` did not include themselves in freestanding.

The author assumes that this is an oversight, since I cannot see any reason for those views to not be in freestanding. As a result, the range adaptor proposed by this paper will be included in freestanding.

(Adding freestanding markers to the two views mentioned above is probably out of scope for this paper, but if LWG decides that this paper should also solve that oversight, the author is happy to comply.)

§ 4. Implementation Experience

The author implemented this proposal in Compiler Explorer. No significant obstacles are observed.

Note that to save on implementation effort, `scan(rng, func)` is simply aliased to `prescan(next(rng.begin()), *rng.begin(), func)`, so they both shares a single underlying view.

§ 5. Wording

The wording below is based on [N4988].

Wording notes for LWG and editor:

- The wording currently reflects Option 3 from the why three adaptors section, with name as `views::scan`, `prescan` and `partial_sum`.
- `views::scan` and `views::prescan` use the same underlying `ranges::scan_view` to save on complexity and duplication of wording.
- Currently, the three views' definition reside just after `views::transform`'s definition and synopsis, due to the author's perception that they are pretty similar.
- The exposition-only concept `scannable` and `scannable-impl` is basically the same as `indirectly-binary-left-foldable` and `indirectly-binary-left-foldable-impl`; the only difference is that `F` is required to be move constructible instead of copy constructible.

Wording questions to be resolved:

- Should the view be called `scan_view` or `prescan_view`?
- Currently, the current sum is cached in the iterator object (as implemented in range-v3). An alternative is to cache the current sum in the view object (as implemented in `tl::ranges`), such that each iterator only needs to hold a pointer to the parent view and an iterator to the current position. Which one should be chosen?
- Two strategy exist to defer `scan` to `prescan`. First (used currently) is to define two constructors for `scan_view` that takes two and three arguments, and have one constructor delegates to the

other. The other approach is to specify that `scan(E, F)` is expression-equivalent to `scan_view(E, *ranges::begin(E), F)`.

- Due to never being a common range, `scan_view` simply have a single `end() const` member that returns `default_sentinel`. Is that the correct way to do this, or should I write two different `end()` function for both cases, with different constraints, but both returns `default_sentinel`? Or should I actually write a `sentinel<Const>` subclass that wraps the iterator?
- The extra `bool IsInit` parameter seems a bit clunky.
- `regular_invocable` or `invocable`? Currently the wording uses the latter (following range-v3), but `transform_view` used the former.
- The return type of `operator*()` seems unorthodox.

§ 5.1. 17.3.2 Header `<version>` synopsis [version.syn]

In this clause's synopsis, insert a new macro definition in a place that respects the current alphabetical order of the synopsis, and substituting `20XXYYL` by the date of adoption.

```
#define __cpp_lib_ranges_scan 20XXYYL // freestanding, also in <ranges>
```

§ 5.2. 26.2 Header `<ranges>` synopsis [ranges.syn]

Modify the synopsis as follows:

```
// [...]
namespace std::ranges {
  // [...]
  // [range.transform], transform view
  template<input_range V, move_constructible F, bool IsInit>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  class transform_view; // freestanding

  namespace views { inline constexpr unspecified transform = unspecified; }

  // [range.scan], scan view
```

```
  template<input_range V, typename T, move_constructible F>
    requires see below
  class scan_view; // freestanding

  namespace views {
    inline constexpr unspecified scan = unspecified; // freestanding
    inline constexpr unspecified prescan = unspecified; // freestanding
    inline constexpr unspecified prefix_sum = unspecified; // freestanding
  }

  // [range.take], take view
  template<view> class take_view; // freestanding

  template<class T>
    constexpr bool enable_borrowed_range<take_view<T>> =
      enable_borrowed_range<T>; // freestanding

  namespace views { inline constexpr unspecified take = unspecified; } //
  // [...]
}
```

Editor's Note: Add the following subclause to 26.7 Range adaptors [range.adaptors], after 26.7.9 Transform view [range.transform]

§ 5.3. 26.7.□ Scan view [range.scan]

§ 5.4. 26.7.□.1 Overview [range.scan.overview]

1. `scan_view` presents a view that accumulates the results of applying a transformation function to the current state and each element.
2. The name `views::scan` denotes a range adaptor object ([range.adaptor.object]). Given subexpressions `E` and `F`, the expression `views::scan(E, F)` is expression-equivalent to `scan_view(E, F)`.

[*Example 1*:

```
vector<int> vec{1, 2, 3, 4, 5};
for (auto&& i : std::views::scan(vec, std::plus{})) {
  std::print("{} ", i); // prints 1 3 6 10 15
```

```
}
```

-- *end example*]

3. The name `views::prescan` denotes a range adaptor object ([range.adaptor.object]). Given subexpressions `E`, `F` and `G`, the expression `views::prescan(E, F, G)` is expression-equivalent to `scan_view(E, F, G)`.

[*Example 2*:

```
vector<int> vec{1, 2, 3, 4, 5};
for (auto&& i : std::views::prescan(vec, 10, std::plus{})) {
  std::print("{} ", i); // prints 10 11 13 16 20 25
}
```

-- *end example*]

4. The name `views::partial_sum` denotes a range adaptor object ([range.adaptor.object]). Given subexpression `E`, the expression `views::partial_sum(E)` is expression-equivalent to `scan_view(E, std::plus{})`.

§ 5.5. 26.7.□.2 Class template `scan_view` [range.scan.view]

```
namespace std::ranges {
  template<typename V, typename T, typename F, typename U>
    concept scannable-impl =  // exposition only
      movable<U> &&
      convertible_to<T, U> && invocable<F&, U, range_reference_t<V>> &&
      assignable_from<U&, invoke_result_t<F&, U, range_reference_t<V>>>;

  template<typename V, typename T, typename F>
    concept scannable =  // exposition only
      invocable<F&, T, range_reference_t<V>> &&
      convertible_to<invoke_result_t<F&, T, range_reference_t<V>>,
             decay_t<invoke_result_t<F&, T, range_reference_t<V>>>> &&
      scannable-impl<V, T, F,
             decay_t<invoke_result_t<F&, T, range_reference_t<V>>>>;

  template<input_range V, move_constructible T, move_constructible F, bool
    requires view<V> && is_object_v<T> && is_object_v<F> && scannable<V, T
```

```
  class scan_view : public view_interface<scan_view<V, T, F, IsInit>> {
  private:
    // [range.scan.iterator], class template scan_view::iterator
    template<bool> struct iterator; // exposition only

    V base_ = V(); // exposition only
    movable-box<T> init_; // exposition only
    movable-box<F> fun_; // exposition only

  public:
    scan_view() requires default_initializable<V> && default_initializable<
    constexpr explicit scan_view(V base, F fun) requires (!IsInit);
    constexpr explicit scan_view(V base, T init, F fun) requires IsInit;

    constexpr V base() const & requires copy_constructible<V> { return base
    constexpr V base() && { return std::move(base_); }

    constexpr iterator<false> begin();
    constexpr iterator<true> begin() const
      requires range<const V> && scannable<const V, T, const F>;

    constexpr default_sentinel_t end() const noexcept { return default_sen

    constexpr auto size() requires sized_range<V>
    { return ranges::size(base_) + (IsInit ? 1 : 0); }
    constexpr auto size() const requires sized_range<const V>
    { return ranges::size(base_) + (IsInit ? 1 : 0); }
  };

  template<class R, class F>
    scan_view(R&&, F) -> scan_view<views::all_t<R>, range_value_t<R>, F, fa
  template<class R, class T, class F>
    scan_view(R&&, T, F) -> scan_view<views::all_t<R>, T, F, true>;
}
```

```
constexpr explicit scan_view(V base, F fun) requires (!IsInit);
```

1. *Effects*: Initializes `base_` with `std::move(base)` and `fun_` with `std::move(fun)`.

```
constexpr explicit scan_view(V base, T init, F fun) requires IsInit;
```

2. *Effects*: Initializes `base_` with `std::move(base)`, `init_` with `std::move(init)`, and `fun_` with `std::move(fun)`.

```
constexpr iterator<false> begin();
```

3. *Effects*: Equivalent to: `return iterator<false>{*this, ranges::begin(base_)};`

```
constexpr iterator<true> begin() const
  requires range<const V> && scannable<const V, T, const F>;
```

4. *Effects*: Equivalent to: `return iterator<true>{*this, ranges::begin(base_)};`

§ 5.6. 26.7.⍰.3 Class template `scan_view::iterator` [range.scan.iterator]

```
namespace std::ranges {
  template<input_range V, move_constructible T, move_constructible F, bool
    requires view<V> && is_object_v<T> && is_object_v<F> && scannable<V, T,
  template<bool Const>
  class scan_view<V, T, F, IsInit>::iterator {
  private:
    using Parent = maybe-const<Const, scan_view>; // exposition only
    using Base = maybe-const<Const, V>; // exposition only
    using RefType = invoke_result_t<maybe-const<Const, F>&, T, range_refere
    using SumType = decay_t<RefType>; // exposition only

    iterator_t<Base> current_ = iterator_t<Base>(); // exposition only
    Parent* parent_ = nullptr; // exposition only
    movable-box<SumType> sum_; // exposition only
    bool is_init_ = IsInit; // exposition only

  public:
    using iterator_concept =
      conditional_t<forward_range<Base>, forward_iterator_tag, input_iterat
    using iterator_category = see below; // present only if Base models for
    using value_type = SumType;
    using difference_type = range_difference_t<Base>;

    iterator() requires default_initializable<iterator_t<Base>> = default;
    constexpr iterator(Parent& parent, iterator_t<Base> current);
```

```
    constexpr iterator(iterator<!Const> i)
      requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

    constexpr const iterator_t<Base>& base() const & noexcept;
    constexpr iterator_t<Base> base() &&;

    constexpr const SumType& operator*() const { return *sum_; }

    constexpr iterator& operator++();
    constexpr void operator++(int);
    constexpr iterator operator++(int) requires forward_range<Base>;

    friend constexpr bool operator==(const iterator& x, const iterator& y)
      requires equality_comparable<iterator_t<Base>>;
    friend constexpr bool operator==(const iterator& x, default_sentinel_t
  };
}
```

1. If `Base` does not model `forward_range` there is no member `iterator_category`. Otherwise, the *typedef-name* `iterator_category` denotes:

- `forward_iterator_tag` if `iterator_traits<iterator_t<Base>>::iterator_category` models `derived_from<forward_iterator_tag>` and `is_reference_v<invoke_result_t<maybe-const<Const, F>&, maybe-const<Const, T>&, range_reference_t<Base>>>` is `true`;
- otherwise, `input_iterator_tag`.

```
constexpr iterator(Parent& parent, iterator_t<Base> current);
```

2. *Effects*: Initializes `current_` with `std::move(current)` and `parent_` with `addressof(parent)`. Then, equivalent to:

```
if constexpr (IsInit) { sum_ = *parent_->init_; }
else {
  if (current_ != ranges::end(parent_->base_)) { sum_ = *current_; }
}
```

```
constexpr iterator(iterator<!Const> i)
```

```
  requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

3. *Effects*: Initializes `current_` with `std::move(i.current_)`, `parent_` with `i.parent_`, `sum_` with `std::move(i.sum_)`, and `is_init_` with `i.is_init_`.

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

4. *Effects*: Equivalent to: `return current_;`

```
constexpr iterator_t<Base> base() &&;
```

5. *Effects*: Equivalent to: `return std::move(current_);`

```
constexpr iterator& operator++();
```

6. *Effects*: Equivalent to:

```
if (!is_init_) { ++current_; }
else { is_init_ = false; }
if (current_ != ranges::end(parent_->base_)) {
  sum_ = invoke(*parent_->fun_, std::move(*sum_), *current_);
}
return *this;
```

```
constexpr void operator++(int);
```

7. *Effects*: Equivalent to `++*this`.

```
constexpr iterator operator++(int) requires forward_range<Base>;
```

8. *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
  requires equality_comparable<iterator_t<Base>>;
```

9. *Effects*: Equivalent to: `return x.current_ == y.current_;`

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

10. *Effects*: Equivalent to: `return x.current_ == ranges::end(x.parent_->base_);`

§ References

§ Normative References

[N4988]
Thomas Köppe. *Working Draft, Programming Languages — C++*. 5 August 2024. URL: https://wg21.link/n4988

[P2760R1]
Barry Revzin. *A Plan for C++26 Ranges*. 15 December 2023. URL: https://wg21.link/p2760r1

§ Informative References

[P1642R11]
Ben Craig. *Freestanding Library: Easy [utilities], [ranges], and [iterators]*. 1 July 2022. URL: https://wg21.link/p1642r11

[P2214R2]
Barry Revzin, Conor Hoekstra, Tim Song. *A Plan for C++23 Ranges*. 18 February 2022. URL: https://wg21.link/p2214r2

[P2322R6]
Barry Revzin. *ranges::fold*. 22 April 2022. URL: https://wg21.link/p2322r6

[P2441R2]

Barry Revzin. *views::join_with*. 28 January 2022. URL: https://wg21.link/p2441r2

[P3117R0]

Zach Laine, Barry Revzin. *Extending Conditionally Borrowed*. 15 February 2024. URL: https://wg21.link/p3117r0

[P3136R0]

Tim Song. *Retiring niebloids*. 15 February 2024. URL: https://wg21.link/p3136r0